Name : Sahil Jakhariya
Div : D15C
Batch : C
Roll No : 63

MLDL Experiment 05

Aim :

Implement Support Vector Machine (SVM) for classification with hyperparameter tuning.

1. Dataset Source :

● **Dataset Name:** Car Price Prediction Dataset

● **Source: Kaggle:** Dataset Repository

● **Load Method:** pandas.read_csv("CarPrice_Assignment.csv")

2. Dataset Description

The Car Price dataset contains technical specifications and features of different cars used to analyze and predict the price category of vehicles.

In this experiment, the continuous price variable is converted into a classification problem by dividing it into two categories:

- 0 → Low Price Car
- 1 → High Price Car

This allows the use of Support Vector Machine (SVM) classification.

Dataset Details:

● Size: 205 samples × 26 features

● Target Variable: price_class

● Target Classes:

- 0 → Low Price Car
- 1 → High Price Car

Features

The dataset contains multiple attributes describing car specifications, including:

○ Symboling
○ Fuel Type ○
Aspiration
○ Number of Doors
○ Car Body
○ Drive Wheel
○ Engine Location
○ Wheel Base
○ Car Length
○ Car Width
○ Car Height
○ Curb Weight ○
Engine Type
○ Number of Cylinders
○ Engine Size

○ Fuel System
○ Bore Ratio
○ Stroke
○ Compression Ratio
○ Horsepower
○ Peak RPM
○ City MPG
○ Highway MPG
○ Price

The dataset contains both numerical and categorical features, which require preprocessing before training the machine learning model.

3. Mathematical Formulation of the Algorithm

Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression tasks. It works by finding the optimal hyperplane that separates different classes with the maximum margin.

A. Hyperplane Equation

In an n-dimensional space, the hyperplane can be represented as:

$$w \cdot x + b = 0$$

Where:

- w = weight vector

- x = input feature vector
- b = bias

The goal of SVM is to find the hyperplane that maximizes the margin between two classes.

B. Margin Maximization

Margin is defined as the distance between the hyperplane and the nearest data points, which are called support vectors.

Maximizing the margin improves the generalization ability of the model.

C. Kernel Trick

When data is not linearly separable, SVM uses kernel functions to map data into a higher-dimensional space where separation becomes easier.

Common kernels used in SVM include:

- Linear Kernel
- Polynomial Kernel
- RBF (Radial Basis Function) Kernel

RBF Kernel Formula:

$$K(x_i, x_j) = exp(-\gamma \mid\mid x_i - x_j \mid\mid^2)$$

The RBF kernel is widely used because it can handle complex non-linear decision boundaries.

4. Algorithm Limitations:

- Computational Cost: Training SVM can be computationally expensive for large datasets.
- Sensitive to Noise: If classes overlap significantly, classification performance may decrease.
- Parameter Selection: Choosing optimal parameters such as C and gamma requires careful tuning.

5. Methodology / Workflow:

The experiment follows these steps:

1. Load Dataset using Pandas
2. Data Preprocessing
   - Convert categorical features into numerical values
   - Create price classification labels
3. Feature Scaling using StandardScaler
4. Train-Test Split (80% training, 20% testing)
5. Hyperparameter Tuning using GridSearchCV
6. Model Training using best parameters

7. Model Evaluation ○ Accuracy ○ Classification Report ○ Confusion Matrix
8. Visualization of results

6. Performance Analysis:

- Expected Accuracy: 85% – 95% depending on preprocessing
- Best Parameters usually observed:
  - C = 10
  - Gamma = 0.01
- Confusion Matrix helps analyze correct and incorrect predictions between high-price and low-price cars.
- The classification report provides precision, recall, and F1-score for model evaluation.

7. Hyperparameter Tuning:

Hyperparameter tuning is the process of selecting the optimal external configuration parameters before training the model.

Grid Search (GridSearchCV)

GridSearchCV is used to test different combinations of hyperparameters.

- Evaluates all parameter combinations

- Uses K-Fold Cross Validation (cv = 5)
- Selects parameters producing the highest validation accuracy

Example:

If we test

- 3 values of C
- 3 values of Gamma

Total models trained:

$$3 \times 3 = 9$$

With 5-fold cross validation:

$$9 \times 5 = 45 \text{ model fits}$$

<u>SVM Hyperparameters Tuned</u>

1. C (Regularization Parameter)

- Controls trade-off between smooth decision boundary and classification accuracy
- High C → Complex boundary → Risk of Overfitting
- Low C → Smooth boundary → Risk of Underfitting

2. Gamma (Kernel Coefficient)

- Determines influence of a single training example
- High Gamma → Tight boundary → Overfitting
- Low Gamma → Smooth boundary → Better generalization

3. Kernel

- RBF kernel is used to capture non-linear relationships in the dataset.

Code And Output:

```
import pandas as pd import numpy as np import matplotlib.pyplot

as plt import seaborn as sns import os from

sklearn.model_selection import train_test_split, GridSearchCV from

sklearn.preprocessing import StandardScaler, LabelEncoder from

sklearn.svm import SVC from sklearn.metrics import

accuracy_score, classification_report, confusion_matrix

print("===== CAR PRICE DATASET - SVM CLASSIFICATION =====\n")

# Show available files print("Files in current

directory:\n") print(os.listdir()) # Load

dataset print("\nLoading Dataset...\n") df =
```

```
pd.read_csv("CarPrice_Assignment.csv")

print("Dataset Loaded Successfully!\n")

print("Dataset Shape:", df.shape)

print("\nFirst 5 rows:\n") print(df.head())

# Convert price to classification (High / Low) median_price =

df['price'].median() df['price_class'] = df['price'].apply(lambda x: 1 if x >

median_price else 0) print("\nPrice classification created (High=1,

Low=0)\n")

# Drop unnecessary columns if

'car_ID' in df.columns:

    df = df.drop('car_ID', axis=1)

# Encode categorical columns

le = LabelEncoder() for col in

df.columns:

 if df[col].dtype == 'object':

df[col] = le.fit_transform(df[col]) #

Features and target

X = df.drop(['price','price_class'], axis=1)

y = df['price_class'] # Feature scaling

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# Train Test Split
```

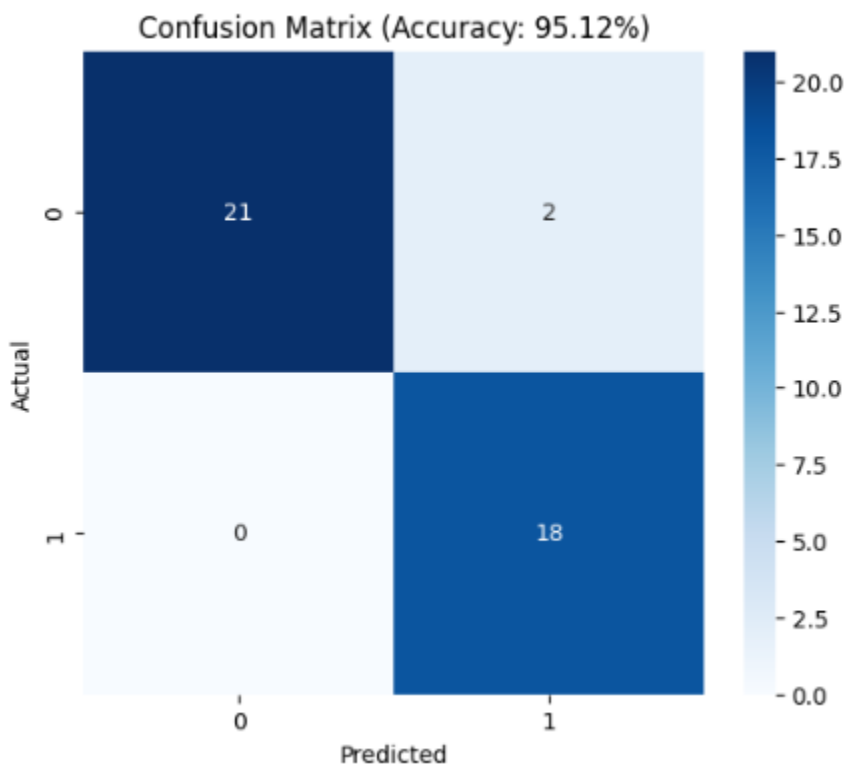
```
X_train, X_test, y_train, y_test = train_test_split(

X_scaled,

    y,

    test_size=0.2,

random_state=42

)

print("Training Data Shape:", X_train.shape)

print("Testing Data Shape:", X_test.shape) #

Hyperparameter tuning param_grid = {

    'C': [0.1,1,10],

    'gamma':[1,0.1,0.01],

    'kernel':['rbf']

}
print("\nStarting GridSearchCV...\n") grid =

GridSearchCV(SVC(),param_grid,cv=5,verbose=1)

grid.fit(X_train,y_train) print("\nBest Parameters

Found:") print(grid.best_params_) # Predictions y_pred

= grid.predict(X_test) print("\n===== MODEL

PERFORMANCE =====\n") accuracy =

accuracy_score(y_test,y_pred)

print("Accuracy:",accuracy) print("\nClassification

Report:\n") print(classification_report(y_test,y_pred))

# Confusion Matrix cm =

confusion_matrix(y_test,y_pred)
```

```
plt.figure(figsize=(6,5))

sns.heatmap(cm,annot=True,fmt='d',cmap='Blues')

plt.title(f"Confusion Matrix (Accuracy:

{accuracy:.2%})") plt.ylabel("Actual")

plt.xlabel("Predicted") plt.show()
```

Output:

```
... ===== CAR PRICE DATASET - SVM CLASSIFICATION =====

Files in current directory:

['.config', 'CarPrice_Assignment.csv', 'sample_data']

Loading Dataset...

Dataset Loaded Successfully!

Dataset Shape: (205, 26)

First 5 rows:

   car_ID  symboling                   CarName fueltype aspiration doornumber  \
0       1          3        alfa-romero giulia      gas        std        two
1       2          3       alfa-romero stelvio      gas        std        two
2       3          1  alfa-romero Quadrifoglio      gas        std        two
3       4          2               audi 100 ls      gas        std       four
4       5          2                audi 100ls      gas        std       four

       carbody drivewheel enginelocation  wheelbase  ...  enginesize  \
0  convertible        rwd          front       88.6  ...         130
1  convertible        rwd          front       88.6  ...         130
2    hatchback        rwd          front       94.5  ...         152
3        sedan        fwd          front       99.8  ...         109
4        sedan        4wd          front       99.4  ...         136

   fuelsystem  boreratio  stroke compressionratio horsepower  peakrpm citympg  \
0        mpfi       3.47    2.68              9.0        111     5000      21
1        mpfi       3.47    2.68              9.0        111     5000      21
2        mpfi       2.68    3.47              9.0        154     5000      19
3        mpfi       3.19    3.40             10.0        102     5500      24
4        mpfi       3.19    3.40              8.0        115     5500      18

   highwaympg    price
0          27  13495.0
1          27  16500.0
2          26  16500.0
3          30  13950.0
4          22  17450.0

[5 rows x 26 columns]
```

```
Price classification created (High=1, Low=0)

Training Data Shape: (164, 24)
Testing Data Shape: (41, 24)

Starting GridSearchCV...

Fitting 5 folds for each of 9 candidates, totalling 45 fits

Best Parameters Found:
{'C': 0.1, 'gamma': 0.01, 'kernel': 'rbf'}

===== MODEL PERFORMANCE =====

Accuracy: 0.9512195121951219

Classification Report:

              precision    recall  f1-score   support

           0       1.00      0.91      0.95        23
           1       0.90      1.00      0.95        18

    accuracy                           0.95        41
   macro avg       0.95      0.96      0.95        41
weighted avg       0.96      0.95      0.95        41
```



Conclusion:

In this experiment, we implemented an **SVM classifier on the Car Price dataset** to classify cars into **high-price and low-price categories**. The model achieved good accuracy, showing that SVM is effective for classification tasks. Hyperparameter tuning using **GridSearchCV** improved the model performance, and **feature scaling using StandardScaler** helped in better optimization. Thus, **SVM with RBF kernel proved to be an effective technique for car price classification.**